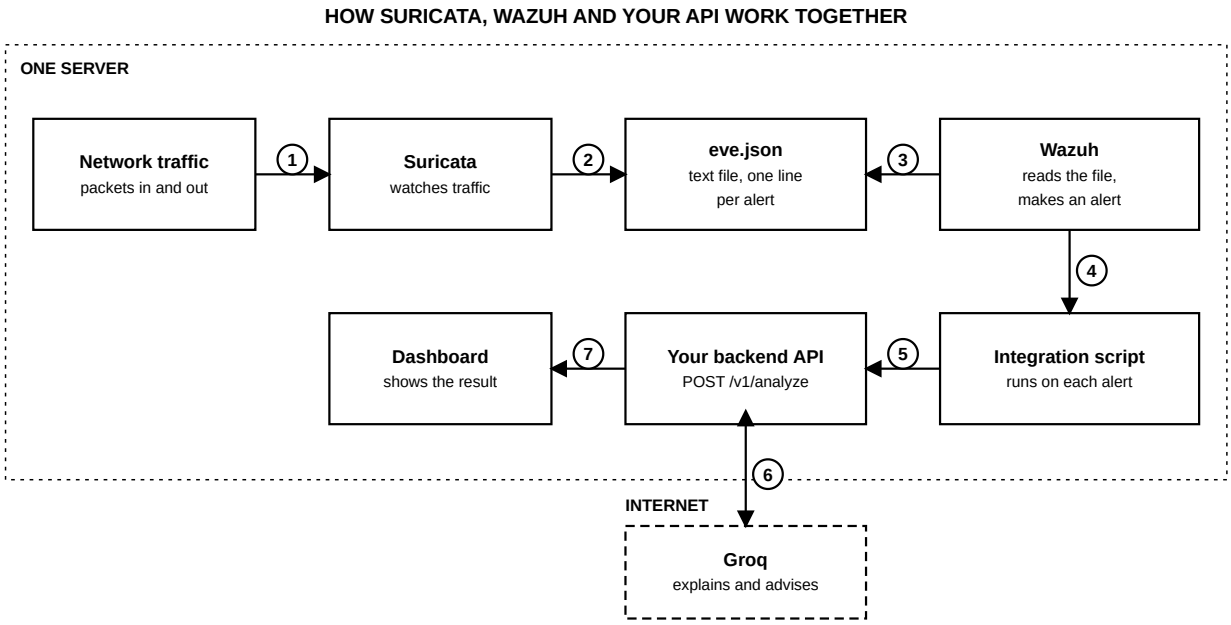
TROJAN: How an Alert Travels, With Examples

Author: Tobi. Date: 1 November 2026. A build guide for the first working path: Suricata and Wazuh raise an alert, our backend sends it to Groq, and the dashboard shows the answer.

All sample data below is made up to show the shape of the real thing. Real Wazuh and Suricata records contain more fields. Anything marked TO CONFIRM must be checked on the real machine before it is relied on.

1. The whole flow

No AI model is trained. Suricata and Wazuh detect with fixed rules. Groq, an AI service on the internet, only explains each alert and suggests what to do. It never blocks or changes anything.



1 Traffic passes the server. 2 A matching rule makes Suricata write a line to eve.json.
3 Wazuh is set to read eve.json. 4 Wazuh runs your script. 5 The script sends the alert to your API.
6 Your API asks Groq. 7 The dashboard shows the alert and Groq's advice. Suricata and Wazuh never talk directly.

2. Who does what

Part	Job	Is it AI?
Suricata	Watches network traffic. Writes a line to eve.json when a rule matches.	No, fixed rules
Wazuh agent	Small program on each computer. Reads logs, watches files, reports to the Manager.	No
Wazuh Manager	Collects everything, applies rules, gives each alert a level from 0 to 15.	No, fixed rules
Integration script	Runs on each Wazuh alert above a chosen level and posts it to our backend.	No
Backend API (FastAPI)	Stores the alert, builds the message for Groq, checks the reply, serves the dashboard.	No
Groq	Reads one alert at a time and writes category, severity and advice.	Yes

Dashboard	Shows the Wazuh result next to the Groq review.	No

3. Step by step with examples

Step 1. Suricata writes a line to eve.json

A test request matches our own TROJAN test rule. Suricata writes one JSON line (shortened here):

```
{"timestamp":"2026-10-29T14:00:02.400332+0200","event_type":"alert",
 "src_ip":"172.26.240.128","src_port":53416,
 "dest_ip":"104.20.23.154","dest_port":80,"proto":"TCP",
 "alert":{"signature_id":1000001,"signature":"TROJAN test rule",
          "category":"","severity":3}}
```

The same alert also appears as a short line in fast.log. This is what the Suricata test showed on 29 October.

Step 2. Wazuh is told to read eve.json

One time setup. Add this block inside the ossec.conf file of the Wazuh Manager (usually /var/ossec/etc/ossec.conf), then restart Wazuh:

```
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>
```

```
sudo systemctl restart wazuh-manager
```

Wazuh now follows that file the way `tail -f` does. Status: TO CONFIRM that the block is not already present from the install.

Step 3. Wazuh raises its own alert

A Wazuh alert is also JSON. For repeated failed logins it looks like this (shortened, made up):

```
{"timestamp":"2026-11-01T03:40:12.000+0200",
 "rule":{"id":"5763","level":10,
         "description":"sshd: brute force trying to get access to the system."},
 "agent":{"id":"000","name":"web-server-1"},
 "data":{"srcip":"203.0.113.45"},
 "full_log":"Failed password for root from 203.0.113.45 port 52118 ssh2"}
```

The rule level (here 10) is a number from 0 to 15. Higher means more serious.

Step 4. The integration script sends it to our backend

Wazuh can run a script for every alert above a level. Add this block to ossec.conf. The name must start with custom-:

```
<integration>
  <name>custom-trojan</name>
  <hook_url>http://localhost:8000/v1/analyze</hook_url>
  <level>7</level>
  <alert_format>json</alert_format>
</integration>
```

Wazuh then runs the script file /var/ossec/integrations/custom-trojan and passes the path of a file holding the alert as the first argument. A minimal version of the script:

```
#!/usr/bin/env python3
import sys, json, urllib.request

alert_file = sys.argv[1]      # Wazuh gives the path to the alert
hook_url   = sys.argv[3]      # Wazuh gives the hook_url from ossec.conf

with open(alert_file) as f:
    alert = json.load(f)

body = json.dumps({"request_id": alert.get("id", "none"),
                   "organisation_id": "demo",
                   "alerts": [alert]}).encode()
req = urllib.request.Request(hook_url, data=body,
        headers={"Content-Type": "application/json"})
urllib.request.urlopen(req, timeout=10)
```

Setup commands for the script file (the group name and argument order are TO CONFIRM on the installed Wazuh version):

```
sudo chmod 750 /var/ossec/integrations/custom-trojan
sudo chown root:wazuh /var/ossec/integrations/custom-trojan
sudo systemctl restart wazuh-manager
```

If the backend is down, the script fails and Wazuh logs the error in /var/ossec/logs/integrations.log. The alert itself is still kept by Wazuh.

Step 5. The backend builds the message for Groq

The backend sends two messages: the fixed system prompt and one user message built from the alert.

System prompt (written once):

```
You are a security expert helping the owner of a small business who is not technical.
You will receive one security alert from Wazuh or Suricata, with a few log lines.
Decide what the alert most likely is, how serious it is for a small business, and what
to do.

Rules:
- Use only what is in the alert. If something is not shown, say you cannot tell. Never
invent details.
- Choose the attack type from this list only: Benign, DoS, DDoS, Reconnaissance, Brute
force FTP,
  Brute force SSH, Web brute force, Cross site scripting, SQL injection, Botnet,
Infiltration,
  Heartbleed, Exploits, Fuzzers, Generic, Backdoor, Shellcode, Worms, Analysis, Other or
unknown.
- Severity must be one of: low, medium, high, critical.
- Write in plain words. Explain any technical term in a few words.
- Suggest steps only. Never say you have blocked or changed anything.
- If the alert looks harmless, say so and give severity low.

Answer with JSON only, in exactly this shape:
{"attack_type":"","severity":"","headline":"","what_happened":"",
 "why_it_matters":"","steps":["",""],"escalate_when":""}
```

User message (built fresh for each alert):

```
Rule: sshd: brute force trying to get access to the system. (level 10)
Time: 2026-11-01 03:40:12
Source address: 203.0.113.45
Machine: web-server-1
Log lines:
Failed password for root from 203.0.113.45 port 52114 ssh2
Failed password for root from 203.0.113.45 port 52116 ssh2
Failed password for root from 203.0.113.45 port 52118 ssh2
```

Step 6. Groq answers

An example of the kind of reply the backend expects (made up, real wording will differ):

```
{"attack_type":"Brute force SSH",
 "severity":"high",
 "headline":"Someone is guessing the password of your server.",
 "what_happened":"One outside address tried to log in as the root account several times
in a few seconds and failed each time.",
 "why_it_matters":"If a guess works, the attacker gets full control of the server.",
 "steps":["Block the address 203.0.113.45 in the firewall.",
          "Check the login log for any successful login from that address.",
          "Turn off password login for root and use keys instead."],
 "escalate_when":"Ask an expert at once if any login from that address succeeded."}
```

The backend checks that every field is present and that attack_type is in the list of 20. If not, it asks Groq once more, then marks the review unavailable.

Step 7. The dashboard shows both results

Wazuh says	**Groq says**
Rule 5763, level 10 sshd: brute force trying to get access to the system. Machine: web-server-1 Source: 203.0.113.45	Brute force SSH, severity HIGH **Someone is guessing the password of your server.** Suggested steps: 1. Block the address. 2. Check the login log. 3. Turn off password login for root. Ask an expert at once if any login from that address succeeded.

The screen says "suggested steps", not "done", because Groq only advises.

4. When something goes wrong

Situation	What happens
Internet is down	The alert still shows on the dashboard, marked not analyzed. The Groq review waits in a queue and is sent when the internet returns.
Groq does not answer in 15 seconds	The backend tries again, up to 3 times, waiting longer each time.
Groq says too many requests (429)	The backend waits as long as Groq asks, then tries again.
Monthly spending limit reached	Groq refuses. The backend stops sending, marks reviews unavailable, and tells the admin.
Groq answers in the wrong shape	Ask once more, then mark the review unavailable.
Backend is down	The script fails and Wazuh logs it. Wazuh keeps the alert.

5. Build order

Test each stage before starting the next. Stage 3 only needs the Groq test script that was already written.

Stage	Build	How to test it
1	Backend with GET /v1/health and POST /v1/analyze. It saves the alert and replies "review pending". No Groq yet.	Open /v1/health in a browser. Post a sample alert with curl and see it stored.
2	Wazuh reads eve.json, and the integration script posts alerts to the backend.	Trigger the TROJAN test rule. Check the alert arrives at the backend.
3	Add the Groq call: system prompt, 15 second timeout, 3 retries, reply check.	Send the sample brute force alert and read the reply.
4	Queue and not analyzed state for when Groq cannot be reached.	Turn off the internet, send 5 alerts, turn it on, see the reviews finish.
5	Dashboard list and detail screen showing both results.	Look at the screen with real alerts.
6	Settings page for the Groq key, with the key stored scrambled and the last 4 characters shown.	Save a key, reload, check only the last 4 characters show and the full key is in no log.

Until stage 6 exists, the backend can read the key from an environment variable named GROQ_API_KEY.

6. Still to confirm

- How many log lines go with each alert. More lines help Groq but cost more and are slower.
- The minimum Wazuh level that is sent to Groq. Level 7 is a starting guess.
- The real field names in a Wazuh alert and a Suricata record on this machine. The examples above are shortened and made up.
- The hourly limit on Groq reviews per business.
- Whether Groq keeps no copy of what is sent (Zero Data Retention).
- Whether the chosen Groq model supports JSON only replies.